STERLING · SBG FUNDING AI GUIDE

Architecture

docs/ARCHITECTURE.md · October 29, 2026

This document explains how Sterling is built, why it is built that way, and what would change to run it for real at SBG's scale. It's written to be read top to bottom before a system design conversation.

---

1. The problem

SBG Funding sells nine financing products to US small businesses. Its website has the pieces of a sales conversation: product pages, two calculators, an FAQ, an application form and a phone number. Nothing ties them together. A visitor has to work out which product fits, whether they qualify and what it might cost, then either apply cold or call during business hours. When they do reach an advisor, the advisor asks the same questions again.

Sterling's job is to be the first ten minutes of an advisor conversation, available all the time:

Goal	How it shows up
Get the visitor to the right product	Guided intake, then products ranked by fit with reasons
Tell them honestly where they stand	Checks against SBG's four published minimums, with what would change a "no"
Give a number	SBG's own calculator model, with a live what-if card
Answer questions accurately	Retrieval over SBG's published content, cited, and "I don't know" when it isn't there
Hand off well	A ticket with the profile, eligibility, a summary and the transcript, plus a reference code

Requirements that shaped the design

- **No made-up facts or figures.** Wrong rates, terms or eligibility in lending is a compliance problem, not just a bad answer.
- **Never imply approval.** Sterling pre-qualifies; only an advisor makes an offer.
- **Consistency.** Two customers with the same profile must get the same answer (fair-lending expectations, and auditability).
- **Sensitive situations go to people.** Hardship, complaints and account questions are routed to humans, deterministically.
- **No sensitive data in the model or logs.** SSNs, card and account numbers are removed on the way in.
- **Fast.** First words on screen in well under a second for guided steps, and streaming for model answers.

- **Keeps working when the model doesn't.** No API key, an outage or a refusal all degrade gracefully.
- **Explainable in a demo.** Every turn's decisions are visible in the "Under the hood" panel.

---

2. Components

```
Browser (React, client-only)
  useChat         holds ClientState; POSTs each turn to /api/chat and reads the NDJSON reply
  Conversation    renders text, cards and sources as events arrive
  FundingFile     the profile panel
  Inspector       the per-turn trace ("Under the hood")
  Voice           Web Speech API: speech in, replies read aloud
  Advisor inbox   polls /api/handoff and updates ticket status

Server (Next.js route handlers, Node runtime)
  /api/chat     ──► orchestrator.handleTurn()
                    ├─ guardrails.checkInput()
                    ├─ flow (next question)
                    ├─ agent.runAgent() ──► Claude Messages API
                    │    └─ tools ──► retriever / eligibility / estimator / escalate
                    └─ offline.offlineTurn()   (fallback)
  /api/handoff  ──► handoff/store (JSON file)
  /api/health   ──► model + index status
```

Module	Responsibility
`lib/chat/orchestrator.ts`	One turn end to end: route the input, call the right subsystem, decide what the flow shows next, always finish with `state` and `done`
`lib/chat/protocol.ts`	The contract between browser and server: `ClientState`, inputs, UI blocks, stream events
`lib/engine/flow.ts`	The guided intake as a state machine: question order per intent, conditional questions, progress
`lib/engine/eligibility.ts`	Published minimums, product rules, ranking and reasons
`lib/engine/estimator.ts`	Port of SBG's funding and line-of-credit calculator scripts
`lib/engine/guardrails.ts`	PII redaction, rule-based routing, injection flag, output check
`lib/agent/agent.ts`	The Claude tool loop with streaming
`lib/agent/tools.ts`	Tool definitions, input validation and implementations
`lib/agent/prompt.ts`	Static system prompt and per-turn `<app_state>`
`lib/agent/offline.ts`	Retrieval-only mode: pattern extraction plus extractive answers
`lib/rag/*`	PDF parsing, chunking, embeddings, BM25 and hybrid retrieval
`lib/handoff/*`	Escalation, tickets, template and AI summaries

3. Life of a turn

3a. A button press (no model call)

The visitor clicks "Payroll" on the first question card.

1. `useChat.send({type: "answer", field: "purpose", value: "payroll"})` POSTs `{state, input}` to `/api/chat`.
2. `handleTurn` validates the client's state (`sanitizeState`: zod-checks the profile, caps history), then `handleAnswer` merges the answer into the profile and emits a `profile` event. The funding file updates.
3. `advance()` asks the flow for the next unanswered field (`nextQuestion`) and emits a `question` UI block.
4. `state` (the updated `ClientState`) and `done` close the stream.

This takes a few milliseconds and costs nothing. Most of the intake works this way.

3b. A typed message (model call)

The visitor types *"I run a roofing company, LLC, 3 years, about $80K a month. Is there a fee to draw on a line of credit?"*

1. **Guardrails first** (`checkInput`). PII is redacted. Routing rules check for a request for a person, a complaint, hardship or account servicing; a match is handled without the model. Injection phrases are flagged in the trace. Here, nothing fires.
2. **Agent loop** (`runAgent`). The request carries the cached system prompt, five tool definitions, the text-only history and, on the newest user message, an `<app_state>` block with the profile, the next guided question, business hours and whether a ticket exists.
3. Claude streams. As soon as it starts a tool call, the UI shows a status line ("Updating your funding file…", "Searching SBG's knowledge base…"). In one step it might call:
   - `update_business_profile({industry: "roofing", businessType: "llc", monthsInBusiness: 36, monthlyRevenue: 80000})`, which is validated with zod and merged. A `profile` event makes the funding file highlight the four new values.
   - `search_knowledge_base({query: "line of credit draw fee"})`, which runs hybrid retrieval and returns numbered passages.
4. Tool results go back in one user message and Claude answers, citing passages as [1]. Text streams to the browser token by token.
5. **Output check** (`checkOutput`) appends a correction if the answer promised approval.
6. The orchestrator emits only the sources the answer actually cited, then calls `advance()`. The intake continues where it left off, now skipping the four fields the visitor already gave.

The stream

The response is newline-delimited JSON, one event per line:

```
{"type":"trace","event":{"t":0,"type":"guardrail","label":"Input checks: clean"}}
{"type":"status","label":"Searching SBG's knowledge base"}
{"type":"profile","profile":{...},"changed":["industry","businessType"],"via":"ai"}
{"type":"text","delta":"Yes. Draws on a business line"}
{"type":"sources","sources":[{"n":1,"title":"Business line of credit","url":"..."}]}
{"type":"ui","block":{"kind":"question","question":{...}}}
{"type":"state","state":{...}}
{"type":"done"}
```

NDJSON over a plain `fetch` stream was chosen over Server-Sent Events or WebSockets because the client sends one request per turn and reads one response. It's simpler than both and works through any proxy that supports chunked responses.

---

4. Key design decisions

Code decides, the model explains

The model never decides eligibility or computes money. `check_eligibility` and `estimate_funding` are tools backed by deterministic code, and the prompt tells Claude to use their outputs verbatim.

Why:

- **Correctness.** Language models are unreliable at arithmetic and at applying thresholds consistently.
- **Consistency and fairness.** The same inputs always produce the same result, which matters under fair-lending rules (ECOA / Regulation B) and for audits.
- **Testability.** The calculator is unit-tested against values computed by hand from SBG's formulas, and eligibility has scenario tests.
- **Change control.** When SBG changes a minimum, it's a one-line change in `lib/sbg/company.ts` with a test, not a prompt tweak to re-verify.

The model's real strengths are understanding messy language, choosing the right tool and explaining results kindly.

The guided flow is a state machine, not a prompt

Question order, conditional questions (invoices for invoice-heavy industries, a repayment event for bridges, home equity when revenue is short) and progress live in `flow.ts`. The model sees the next question in `<app_state>` and is told not to ask it, because the app will show it as a card. That gives predictable UX, works without the model, and makes the intake testable. The model handles the part a state machine can't: free-text answers that fill several fields at once.

A stateless server; the client holds the conversation

Each request carries the whole `ClientState` (profile, intent, text history capped at 24 turns, ticket id), and the response returns the updated state. The server stores nothing per session except handoff tickets.

- **Pros:** any server instance can handle any turn, so it scales horizontally without sticky sessions or a session store. Deploys don't drop conversations. It's simple.
- **Cons:** the client could tamper with state, so `sanitizeState` revalidates everything and limits sizes. Eligibility is recomputed from the profile on the server and never trusted from the client. Payloads grow with history, which is why it's capped and text-only.
- **At scale:** keep the stateless pattern, but persist conversations server-side (keyed by session id) for analytics, QA review and handoff context.

Only text history is sent back

Within a turn, the agent loop echoes Claude's full content (thinking blocks with signatures, tool calls) as required. Across turns only the visible text is kept. Old tool results are large and stale. The profile carries the facts forward in `<app_state>` instead, and retrieval runs again when needed.

Prompt caching and where per-turn context goes

The system prompt is identical on every request and marked with `cache_control`. Because tools come before the system prompt in the cached prefix, the tool definitions are cached too. Everything that changes per turn (profile, time, next question) goes into `<app_state>` on the newest user message. Putting it in the system prompt would invalidate the cache every turn.

Hybrid retrieval with a confidence gate

Dense embeddings handle paraphrase ("I can't cover payroll" matches "cover payroll gaps"). BM25 handles exact tokens: "844-284-2725", "UCC", "7(a)", "Mission Capital LLC". Reciprocal Rank Fusion merges them by rank, so their different score scales don't matter. On the eval set hybrid gets hit@3 93.5% against 89.1% for dense alone. If the best cosine similarity is below 0.33 the confidence is "low" and Sterling declines and offers a specialist. Off-topic questions were flagged 100% of the time.

Deterministic routing for sensitive messages

Requests for a person, complaints, hardship and account servicing are matched by rules before the model sees the message. Rules are auditable, instant and can't be talked out of escalating. The model can also escalate through `escalate_to_human` for the cases rules can't see, such as an unanswerable question or a complex deal. Both paths share `escalate()`. Servicing tickets carry no product suggestions: someone who can't make a payment shouldn't be handed new financing to pitch.

Warm handoff

A ticket holds the reason and priority (complaints and hardship are high), the profile, whether the visitor meets the minimums, best-fit products (sales leads only), the redacted transcript and a summary. The template summary is written immediately so the ticket is useful at once. A model-written summary

replaces it after the response is sent, using Next's `after()`, so the visitor never waits for it. The reference code (`SBG-` plus six characters from an alphabet without 0/O or 1/I) can be read over the phone. Business-hours awareness (New York time) sets expectations: "available now" or "back tomorrow at 9am ET".

Graceful degradation

Failure	Behavior
No API key	Retrieval-only mode: same flow, eligibility, estimates, guardrails and handoffs; extractive answers; regex profile extraction
API error, overload or timeout	The SDK retries twice; if text had already streamed, a short apology and a specialist offer; otherwise retrieval-only mode for that turn
Safety refusal	Server-side fallback to another model (beta); if still refused, a polite redirect to the phone number, and tool calls from a refused turn are never run
Malformed tool input	Validation error returned to the model as `is_error` so it can correct itself; unparseable streamed input retries the step
Response cut off by max_tokens	Tool calls in a truncated response are not executed

---

5. The RAG pipeline in detail

Source. `knowledge/source/*.md`: 9 documents and 73 sections, paraphrased from SBG's public pages. Every section names its source URL. `npm run kb:pdf` renders them to a 33-page PDF, which stands in for "SBG's documents".

Ingest (`npm run ingest`):

1. **Parse.** `pdfjs-dist` extracts text runs with positions and font sizes. Runs are grouped into lines by baseline. Headings are recovered from font size (title 22pt, section 15pt, subsection 12pt). The cover page is skipped, and wrapped headings, wrapped source lines and hyphenated line breaks are repaired.
2. **Chunk.** Structure-aware: a chunk never crosses a heading. Sections longer than 150 words are split at paragraph or bullet boundaries, and the last item of a chunk is repeated at the start of the next (up to 40 words) so a list's context isn't lost.
3. **Contextualize.** The text that gets embedded is `"Section > Title\nPassage"`, so a passage like "Up to 90% of value" carries "Equipment financing" with it.
4. **Embed.** all-MiniLM-L6-v2 runs locally on ONNX Runtime, mean-pooled and normalized to 384 dimensions. It's free, private, and fast enough on CPU for 105 chunks plus one query per search.
5. **Index.** `data/kb-index.json` holds chunks, metadata and vectors. At this size brute-force cosine over 105 vectors takes well under a millisecond.

Query (`search()`): embed the query, compute cosine against every chunk, score BM25 over title and text, rank both, fuse with weighted RRF (`score = 1/(60 + dense_rank) + 0.3/(60 + bm25_rank)` over each ranker's top 20), return the top k with their scores, and compute confidence from the best cosine.

---

6. Data model

```
Profile {            // every field optional; filled by cards or by the model
  purpose, amount, timeline,                // the need
  businessType, industry, timeInBusiness,   // the business
  monthlyRevenue, creditScore,              // the financials
  hasInvoices?, repaymentEvent?, ownsHome?  // asked only when relevant
}

ClientState { sessionId, profile, intent?, history[], resultsKey?, estimateKey?, ticketId? }

Ticket {
  id: "SBG-XXXXXX", status: new | callback_requested | contacted | closed,
  reason, priority: high | normal, summary, summarySource: ai | template,
  profile, meetsMinimums, topMatches[], transcript[], contact?
}
```

`resultsKey` and `estimateKey` are fingerprints of the profile. The same results aren't shown twice, but changing an answer brings them back.

---

7. Taking it to production

What changes, in roughly the order I'd do it:

Integrations

- Tickets go to SBG's CRM (Salesforce or HubSpot) through its API instead of a JSON file. High-priority tickets also post to the advisors' Slack or Teams channel.
- "Apply" links pre-fill the application with the captured profile, with consent, so the funnel doesn't restart.
- Existing customers sign in, so Sterling can answer account questions (balance, payoff) through read-only APIs, behind identity verification.

Data and retrieval

- Persist conversations (Postgres) with a retention policy and PII redaction at write time, for QA review, analytics and evals.
- Re-ingest automatically when content changes, triggered by a CMS webhook, and version the index so answers can be traced to a content version.
- At thousands of documents, move vectors to pgvector or a vector database with an approximate nearest-neighbor index (HNSW), and add a reranker over the top 20.

Quality

- Build the eval sets from real (redacted) conversations, not hand-written ones. Run them in CI on every prompt, model or knowledge change, and block regressions.
- Score groundedness: does every factual sentence trace to a cited passage? Use an LLM grader calibrated against human labels.
- Sample live conversations weekly for human review.

Safety and compliance

- Staff SSO on the inbox; rate limiting and bot protection on `/api/chat`.
- Compliance review of all customer-facing language. Commercial financing disclosure laws (for example New York's and California's) govern *offers*. Sterling shows estimates and must keep saying so.
- Fair lending: industry and business data only, no protected characteristics, and the deterministic engine documented for audit.
- Consent for callbacks (TCPA) on the callback form.

Operations

- Metrics: conversations started, intake completion, apply-click rate, escalation rate by reason, time to first token, fallback rate, retrieval confidence distribution, cost per conversation, and advisor feedback on handoff quality.
- Alerts on error rate, fallback rate and latency.
- A/B test entry points (homepage widget vs. a dedicated page) and the intake order.

Scale. The server is stateless, so it scales horizontally behind a load balancer. The real limits are the model API's rate limits and cost. Guided steps never call the model, and cached prompts make model turns cheaper. Background work (AI summaries) moves to a queue with retries.

Channels. The orchestrator doesn't know about HTML. The same turn logic can sit behind SMS, WhatsApp or a phone voice agent with a different renderer for UI blocks.